

\$ git clone

⇒ copying the files in the git repo to the laptop.

\$ git clone "url of repo".

for checking

\$ ls

⇒ copying the files to a new folder.

\$ git clone "url of repo" newfolder.

folder name.

\$ ls.

\$ cd newfolder / repo name /.

\$ cat foldername.extension.

⇒ But bash can't read other than texts;

\$ explorer.

⇒ opens the file folder in file explorer.

\$ start filename.docx

⇒ opens the document in word now.

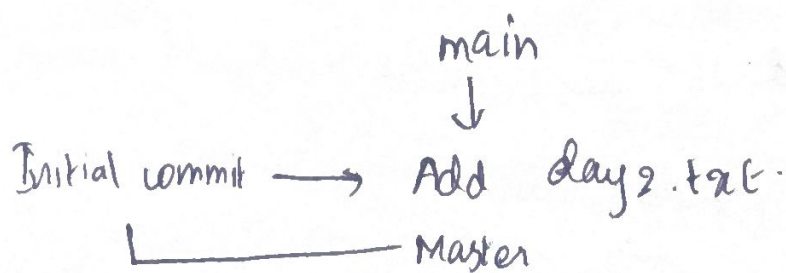
⇒ check whether there exists master branch.

\$ git checkout master.

⇒ To convert to master.

\$ git branch -m master-

⇒



↳ only Initial commit - Initial.txt

git checkout master.

vi gje.txt

add & commit there itself

git checkout main.

merge master to main.

Rebase git checkout main

git log

clear

ls

vi rebasefile

ls

git add rebasefile

git status

git commit -m "rebase practise"

ls

git status

git log

git checkout master

git log

git rebase main.

git log

ls

main
A - B

C - D feature.

Rebase = move your branch's
commits to start
from the latest
commit of another
branch.

⇒ Here one file is added and
committed (rebasefile) in
main and rebased in
master branch

Stash

⇒ Temporarily stores uncommitted
changes so that we can work
on something else with a
clean working directory.

\$ git stash -u

⇒ Save tracked +
untracked files.

\$ git stash list

⇒ see what you have
stashed.

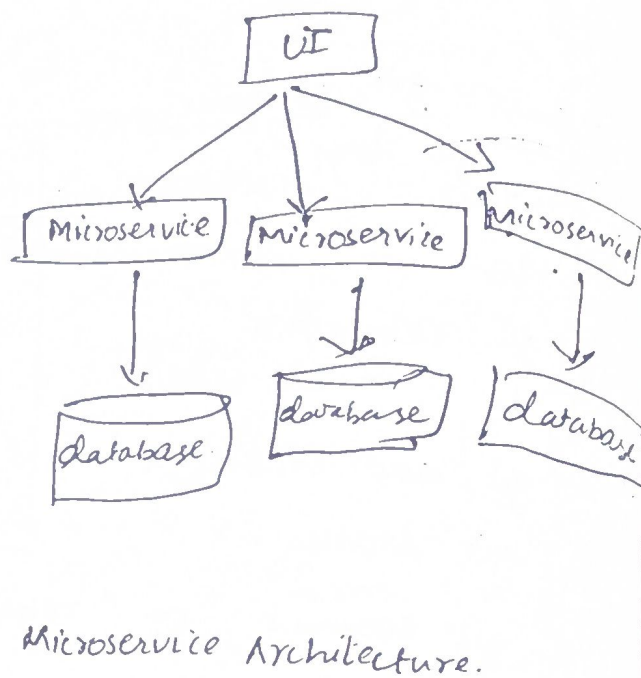
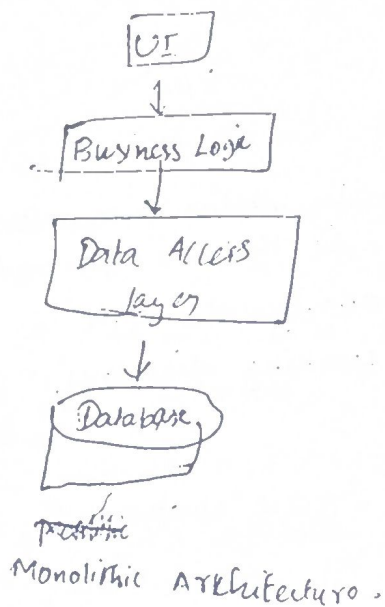
\$ git stash pop

⇒ Restore the latest stash
and remove it from stash

\$ git stash apply

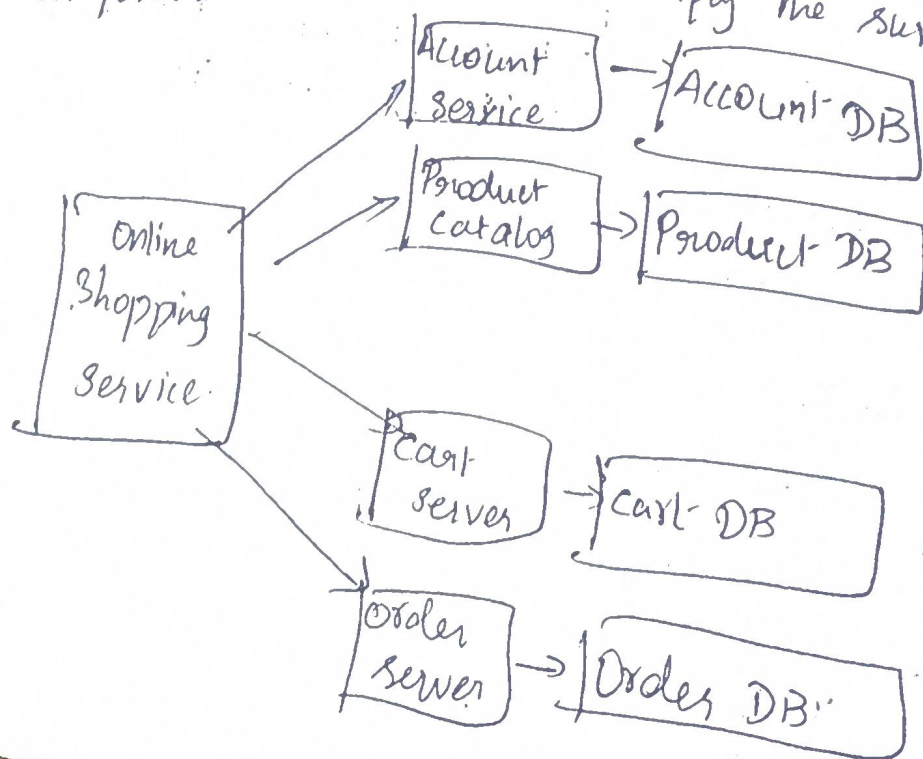
⇒ Restore the stash but keep
it in the stash list.

Problem Statement:
 The university wanted to improve the way they architecture applications, by moving to a microservices base architecture for their applications.



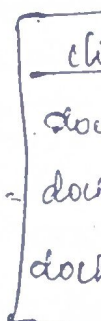
Microservices

The idea behind microservices is that some types of applications become easier to build and maintain when they are broken down into smaller, composable pieces which work together. Each component is developed separately and the application is then simply the sum of its constituent components.



For exam
 microservice
 processing

Container



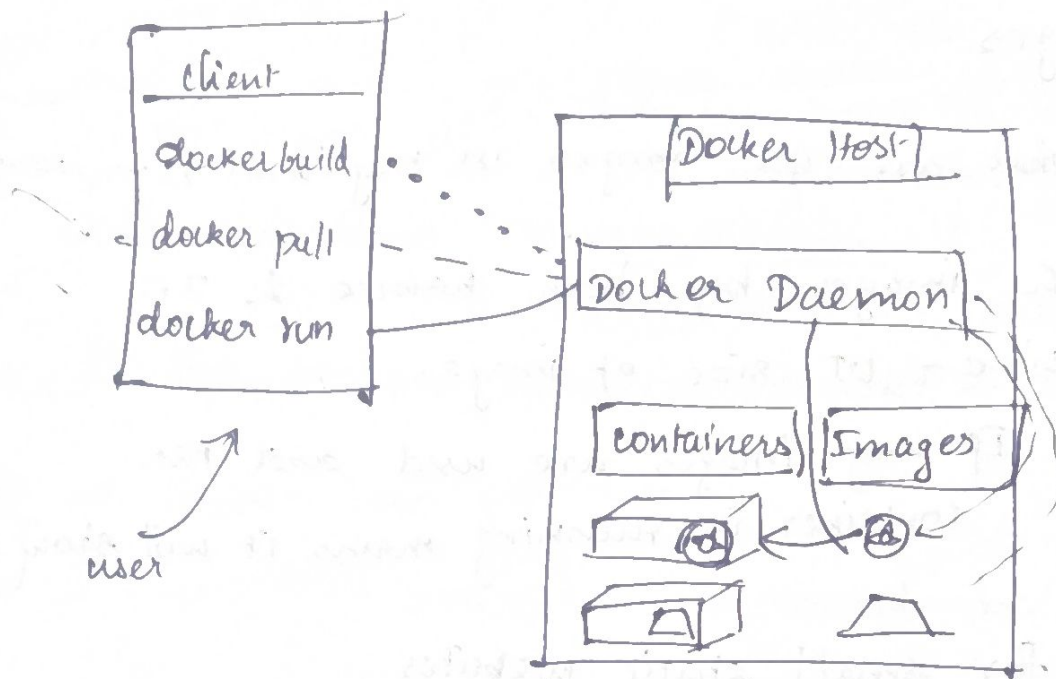
user

Windows
 application

For example imagine an online shop app with separate microservices for user-accounts, product-catalog order-processing and shopping carts.

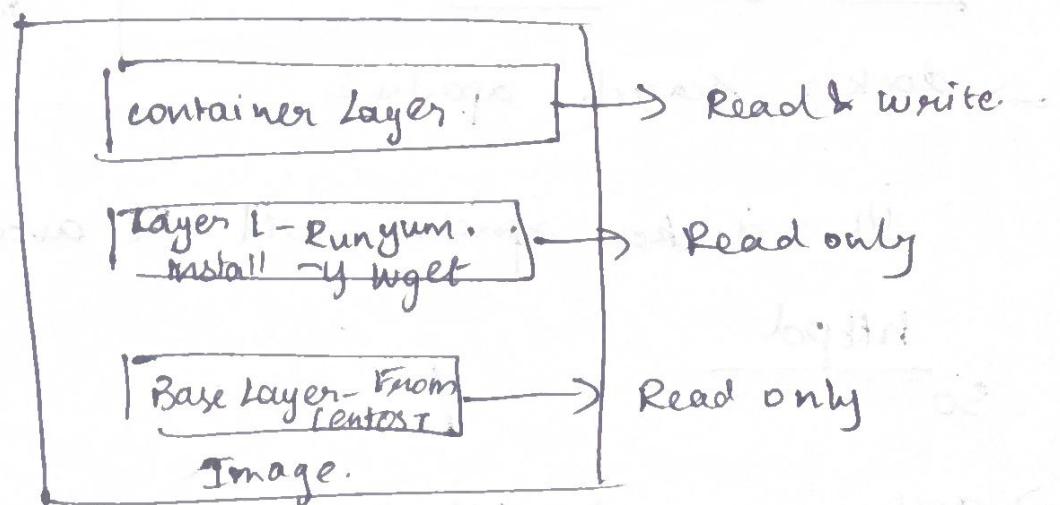
Containerization - Docker

Docker Architecture and Components



Windows subsystem Linux (WSL) - To run linux applications in windows - feature provided by windows.

Docker Images.



⇒ docker - v

⇒ docker search node

⇒ docker pull node ~~se to put~~

* to pull the required image from docker hub to local repository.

⇒ docker images

* To see what are the images in my local repository

★ We can view the images by both name & ID.

Content size - UI size of image.

Extra - If any images are used and the container is running means it will show.

web servers

{ Apache - for small static websites.

{ Nginx - for large dynamic websites which contains many web pages.

→ It handles traffic efficiently.

To host static website.

⇒ docker search apache2

docker was built by reddit distribution

in docker apache2 will be available as.

so httpd

⇒ docker search httpd

⇒ docker pull httpd

★ To launch a container.

⇒

1) Create a container launches only we have to explicitly run

⇒ docker create httpd.
image name.

2) To view the container i have created.

⇒ docker ps -a.

↓ (Here status would be all (shows both created & running containers) created only)

3) To start it

⇒ docker start nervouslichterman.

container name or container ID.

If the container name is returned — container started.

⇒ docker ps -a.

(Here status would be the seconds it was started ago).

4) To list only the running containers

⇒ docker ps